

Evaluating Contextual Embeddings For FastText Classification:

A Comparative Study

Feliciann Elliot, Hilton Sairus, Daryl Nelson

Abstract

The advent of contextualized language models like BERT and Sentence-BERT (SBERT) has set new standards in numerous NLP tasks. This paper investigates whether these advanced embeddings can improve the performance of FastText, a simple and efficient linear model for text classification. Our research evaluates whether contextualized embeddings improve performance over traditional n-gram-based FastText representations while considering computational trade-offs. We propose a hybrid tokenization method that transforms continuous BERT and SBERT embeddings into discrete, symbolic tokens compatible with FastText by employing Principal Component Analysis (PCA) and K-means clustering. Our methodology is rigorously evaluated across four benchmark datasets (AG News, Yelp Review Polarity, Yahoo Answers, and Amazon Reviews) comparing classification performance and computational cost.

Contrary to expectations, experimental results across multiple benchmark datasets reveal that although contextual embeddings offer richer semantic representations, they do not consistently outperform raw text FastText models. Our results indicate that raw text with standard n-grams consistently outperforms or matches the performance of contextual embedding variants, while being vastly more computationally efficient. We conclude that for large-scale text classification with FastText, the

significant overhead of generating and processing contextual embeddings is not justified by corresponding gains in accuracy, affirming the continued effectiveness of traditional tokenization for this class of model.

1. Introduction

Recent advances in natural language processing (NLP) have demonstrated the power of contextual embeddings in enhancing the semantic understanding of text. Models such as BERT (Devlin et al., 2019) and Sentence-BERT (Reimers & Gurevych, 2019) have redefined text representation by capturing bidirectional context. However, integrating these embeddings with shallow classifiers like FastText remains an open question.

FastText is a popular text classification model known for its simplicity, speed, and effectiveness, particularly with large datasets. Its standard approach relies on bag-of-n-grams features, which, while efficient, lack deeper semantic understanding.

This research bridges these two paradigms by exploring a critical question: Can the integration of contextual embeddings from BERT and SBERT improve FastText's classification performance, and what are the computational trade-offs associated with this enhancement? To answer this, we design and implement a hybrid framework that converts continuous embeddings

into a tokenized format processable by FastText. We then conduct a comprehensive experimental evaluation comparing the performance and efficiency of FastText models using raw text, BERT-derived tokens, and SBERT-derived tokens across multiple datasets.

2. Background

2.1 This paper is related to and inspired by the Bag of Tricks for Efficient Text Classification paper. FastText is introduced as a highly efficient text classification method that extends linear models with bag of words and bag of n-grams representations to capture partial word order without the computational cost of full sequence models. Sentences are represented by averaging word and n-gram embeddings, which are then fed into a linear classifier with a rank-constrained weight matrix, trained using stochastic gradient descent and hierarchical softmax for large output spaces. Using n-grams (bigrams or higher) significantly improves accuracy by incorporating local word order, while feature hashing keeps memory usage low. Experiments show that fastText achieves performance comparable to deep learning approaches on sentiment analysis datasets, slightly outperforming character-level CNNs and RNNs, and competitive with very deep CNNs, while training orders of magnitude faster on CPUs. On large-scale tag prediction with over 300K classes, fastText outperforms baselines and similar linear models, achieving a 46% precision at 1 with bigrams and completing training in under 14 minutes, demonstrating that shallow models with n-gram features can rival deep models in both speed and accuracy.

2.2 FastText

FastText, developed by Facebook AI Research, is a lightweight model for text classification. It represents documents as a bag of n-gram features and uses a shallow neural network for

classification. Its efficiency and competitive performance on large-scale tasks have made it a standard baseline in text classification benchmarks. Despite its simplicity, FastText has shown strong performance in large-scale and low-resource settings

2.3 Contextual Embedding Models

- BERT (Bidirectional Encoder Representations from Transformers): A transformer-based model that generates dynamic, context-sensitive embeddings for each token in a sentence. While powerful, its native output is a set of token embeddings, making direct sentence-level comparison computationally expensive.

- Sentence-BERT (SBERT): An extension of BERT fine-tuned using a siamese network structure to produce semantically meaningful, fixed-sized sentence embeddings. SBERT is optimized for tasks like semantic similarity, where it significantly reduces computational time compared to BERT while maintaining high performance.

3. Methodology

3.1 Experimental Pipeline

Our experimental framework consists of five core components:

1. Embedding Extraction: Raw text is processed through either bert-base-uncased (extracting the [CLS] token embedding) or SBERT (all-MiniLM-L6-v2) to generate contextual embeddings.

2. Hybrid Embedding Tokenization: A custom HybridEmbeddingTokenizer class converts continuous embeddings into discrete tokens. This involves:

- Dimensionality Reduction: PCA reduces the embedding dimensions (BERT: 768→64; SBERT: 384→64).

- Clustering: Faiss K-means clustering identifies 4,096 centroids in the reduced space.

- Token Generation: Each embedding is soft-assigned to its top-k nearest clusters, and its PCA dimensions are discretized into bins. The final token sequence is a concatenation of cluster tokens and PCA bin tokens.

3. Model Training: Supervised FastText models are trained on three input types: (1) Raw text (baseline), (2) BERT-tokenized, and (3) SBERT-tokenized. Hyperparameters (epoch=5, lr=0.1) were kept consistent.

4. Evaluation: Models are evaluated on held-out test sets using macro-averaged Accuracy, Precision, Recall, and F1-score. Embedding and training times are recorded.

5. Multi-Dataset Automation: The entire pipeline is executed across four datasets to ensure robustness and generalizability.

3.2 Datasets

We used four public benchmark datasets covering topic classification and sentiment analysis:

- AG News: Topic classification with four news categories.
- Yelp Review Polarity: Binary sentiment classification.
- Yahoo Answers: Multi-class question categorization.
- Amazon Reviews: Product sentiment classification.
- Amazon Customer Review Polarity: Binary sentiment classification.

These vary in size, complexity, and vocabulary.

3.3 Hyperparameters

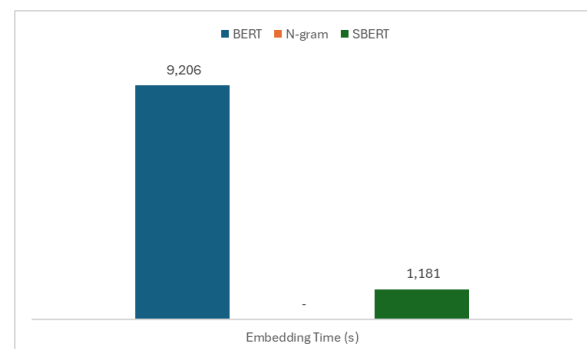
Key hyperparameters for the hybrid tokenizer were n_clusters=4096, pca_dim=64, n_bins=10 and top_k=50 for BERT and 30 for SBERT. This allows multiple cluster memberships per embedding for soft tokenization; SBERT embeddings are lower-dimensional, so fewer clusters are used.

FastText was trained with wordNgrams=2 for raw text and wordNgrams=1 for tokenized inputs.

4. Results and Discussion

4.1 Embedding Time Efficiency

SBERT demonstrated a substantial speed advantage over BERT. For instance, on the AG News dataset, BERT required 248 seconds for embedding generation, while SBERT required only 45 seconds, a 5.5x reduction. On the large Amazon Reviews dataset, BERT's embedding time exceeded 15,000 seconds, compared to approximately 2,400 seconds for SBERT.



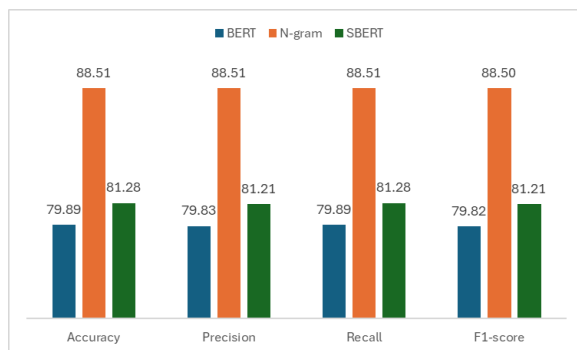
4.2 Classification Performance

The performance results revealed a surprising trend. The raw text FastText model consistently achieved the highest accuracy across most

datasets, outperforming both contextual embedding variants by 2-7% on AG News, Yelp, and Amazon.

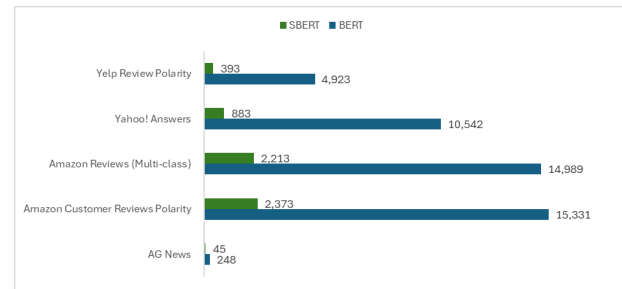
- SBERT vs. BERT: SBERT generally outperformed BERT, especially on larger, noisier datasets like Yahoo Answers (73.08% vs. 56.98%). On smaller datasets like AG News, SBERT also showed an advantage (88.66% vs. 84.24%). However, in some cases (Yelp, Amazon), BERT slightly outperformed SBERT, suggesting token-level context can be beneficial for certain structured texts.

- Contextual vs. Raw Text: Despite their semantic richness, neither BERT nor SBERT tokenization could consistently surpass the simple raw text baseline.

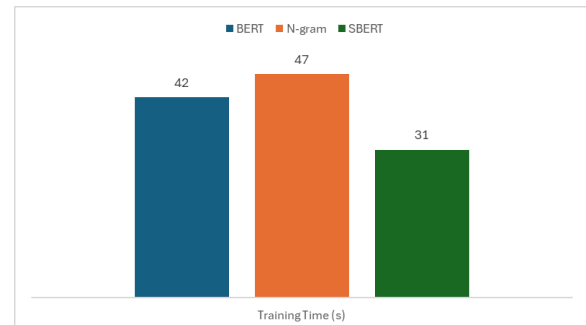


4.3 Efficiency vs. Accuracy Trade-Off

The core finding of this study is the trade-off between computational cost and predictive gain. While contextual embeddings provide richer representations, they introduce significant preprocessing overhead. Raw text processing, requiring no embedding generation, proved to be the optimal choice, offering the best balance of high accuracy and minimal computational cost.



Embedding times



5. Conclusion

This study demonstrates that contextual embeddings (BERT or SBERT) offer minimal benefit when used with FastText classifiers. Raw text processing achieves the strongest balance between predictive performance and computational efficiency across multiple large-scale datasets. These findings suggest that shallow embedding models, such as FastText, are highly effective with traditional tokenization and n-grams, and that the computational cost of contextual embeddings is often not justified.

The efficiency versus accuracy trade off therefore favors raw text models for practical applications requiring scalability and speed.

6. Limitations

This study has several limitations:

- Linguistic: All datasets were in English; multilingual effects were not examined.

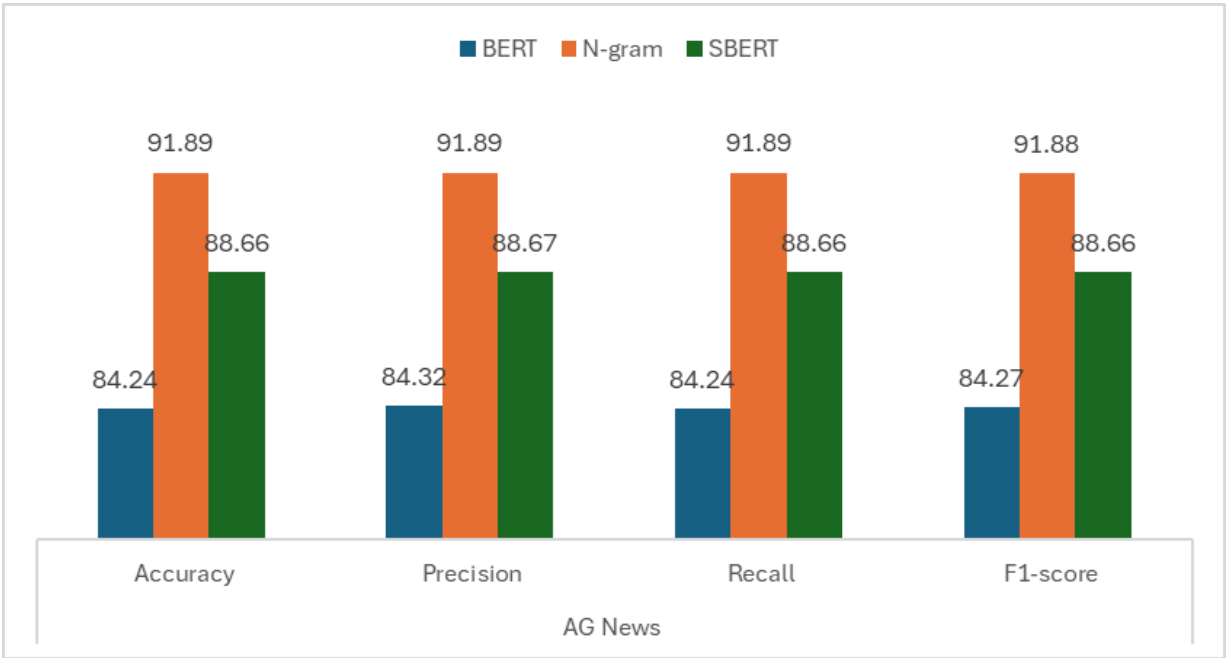
- Metric Scope: Evaluation was based on macro-averaged metrics; micro-averaged or per-class analysis might reveal nuanced performance differences.
- Data Scale: Low-resource or small-data settings were not explored, where contextual embeddings may have a different impact.
- Embedding Flexibility: Only frozen and sentence-level embeddings were tested; further investigation could explore fine-tuning BERT or hybrid shallow-deep models.

7. References

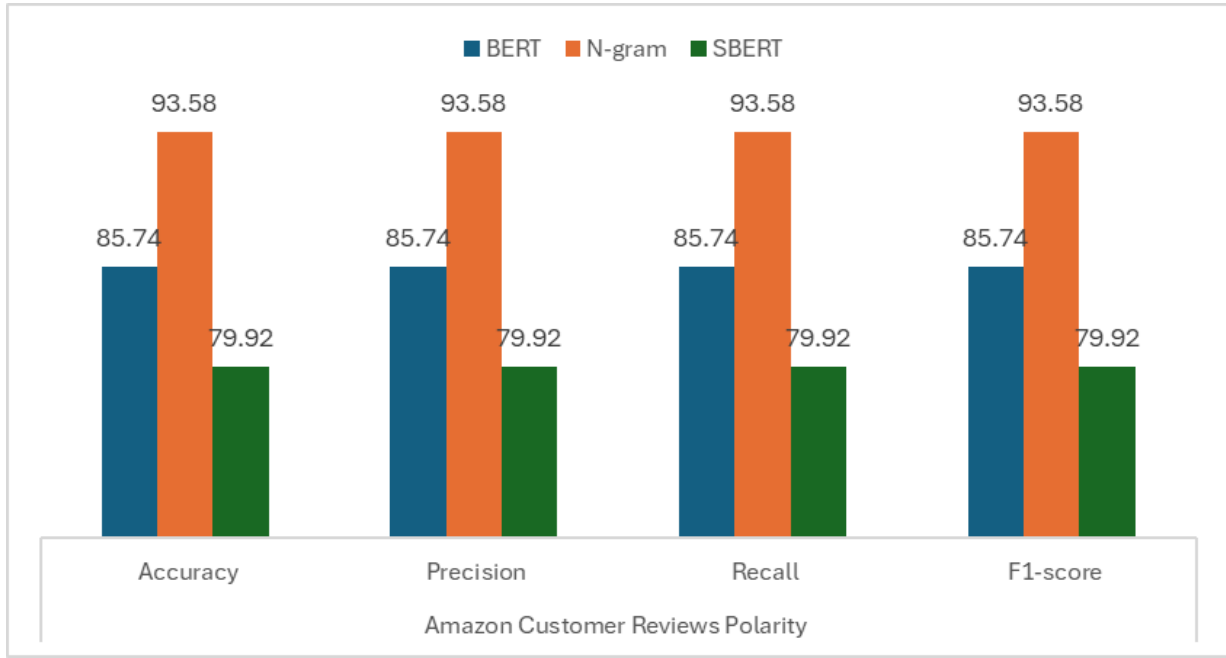
1. Joulin, A., Grave, E., Bojanowski, P., & Mikolov, T. (2017). Bag of Tricks for Efficient Text Classification. Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics.
2. Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. Proceedings of NAACL-HLT.
3. Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing.
4. AG News Dataset
(http://groups.di.unipi.it/~gulli/AG_corpus_of_news_articles.html)
5. Yelp Review Polarity Dataset
(<https://www.yelp.com/dataset>)
6. Yahoo Answers Dataset
(<https://www.kaggle.com/datasets>)
7. Amazon Reviews Dataset
(<https://www.kaggle.com/datasets>)

8. Amazon Reviews Polarity Dataset
(<https://www.kaggle.com/datasets>)

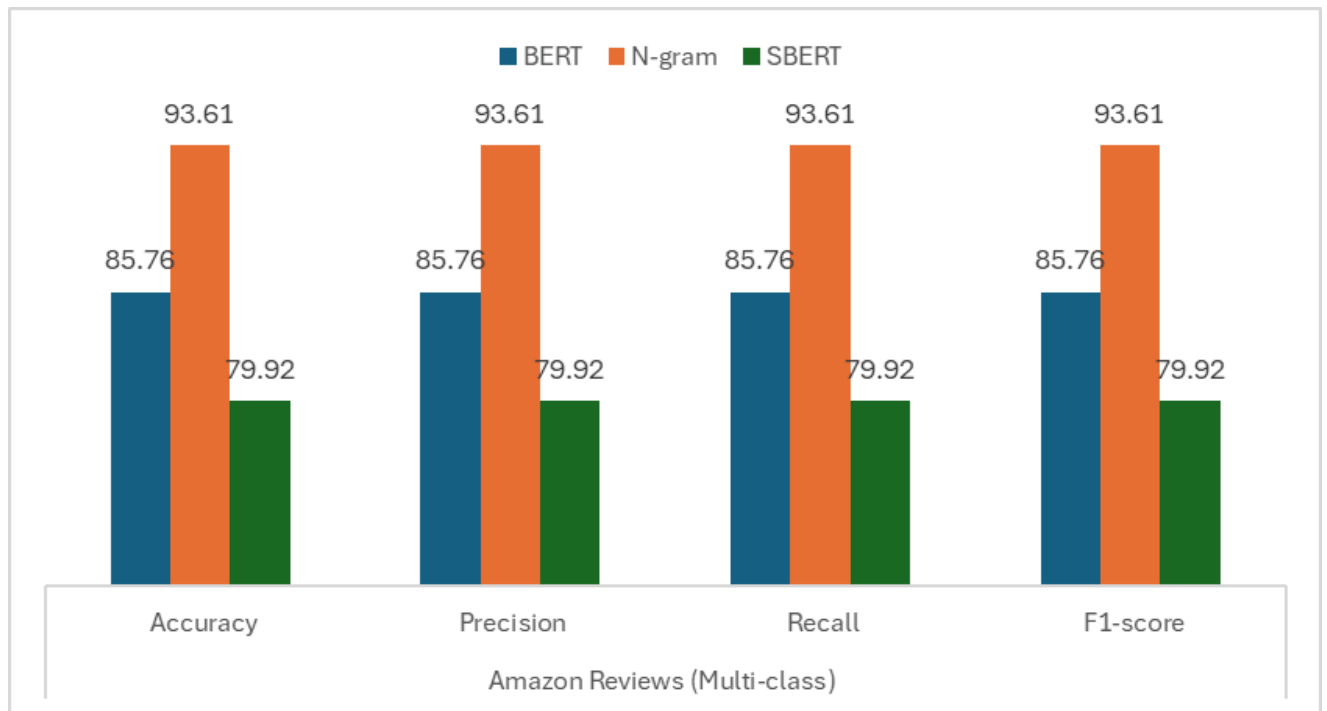
Performance on **AG News** dataset By Model Type



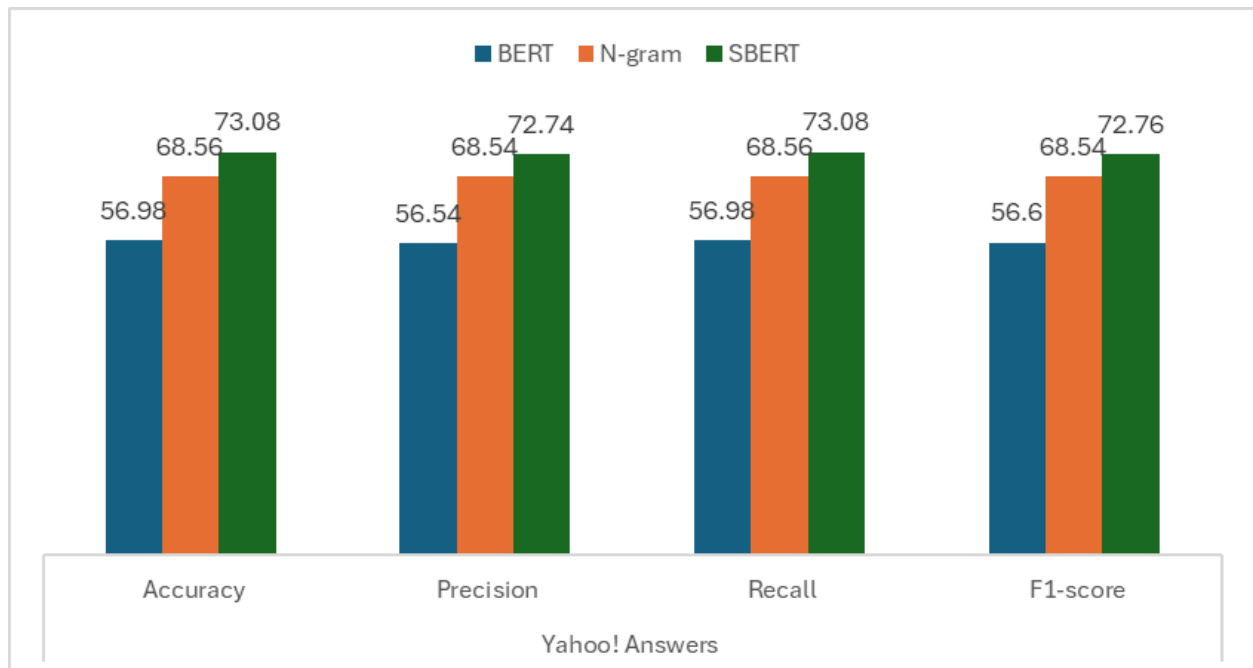
Performance on **Amazon Customer Reviews Polarity** dataset By Model Type



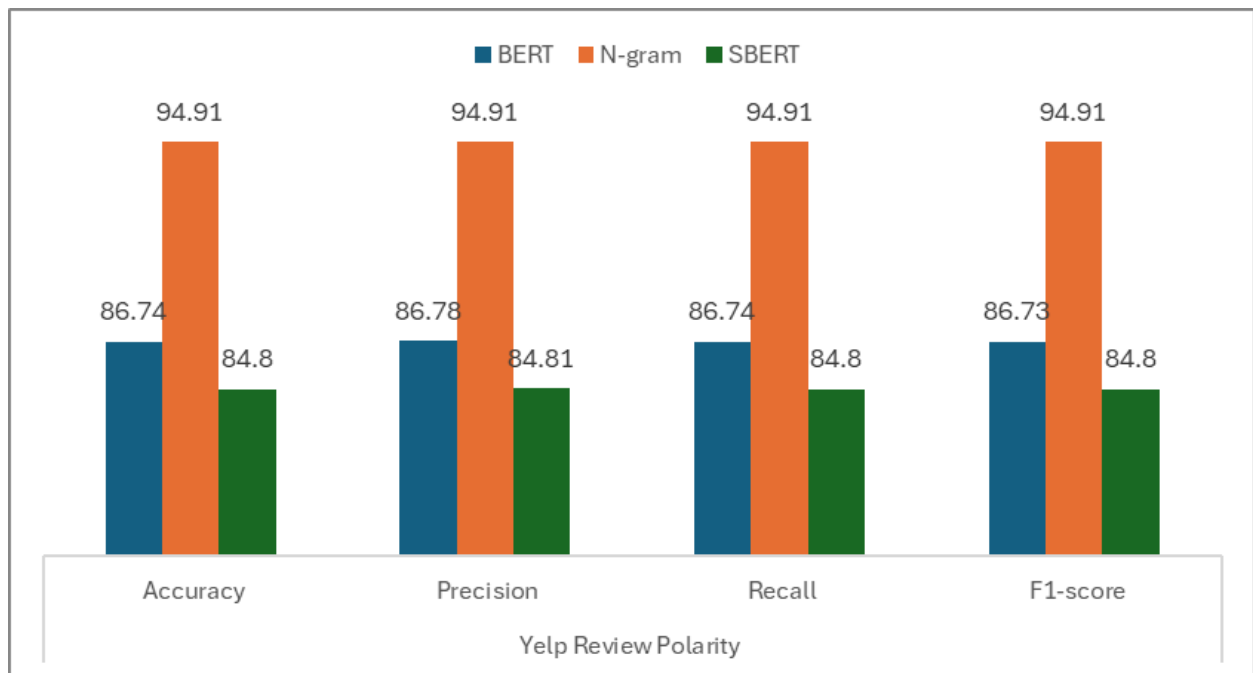
Performance on **Amazon Reviews (Multi-class)** dataset By Model Type



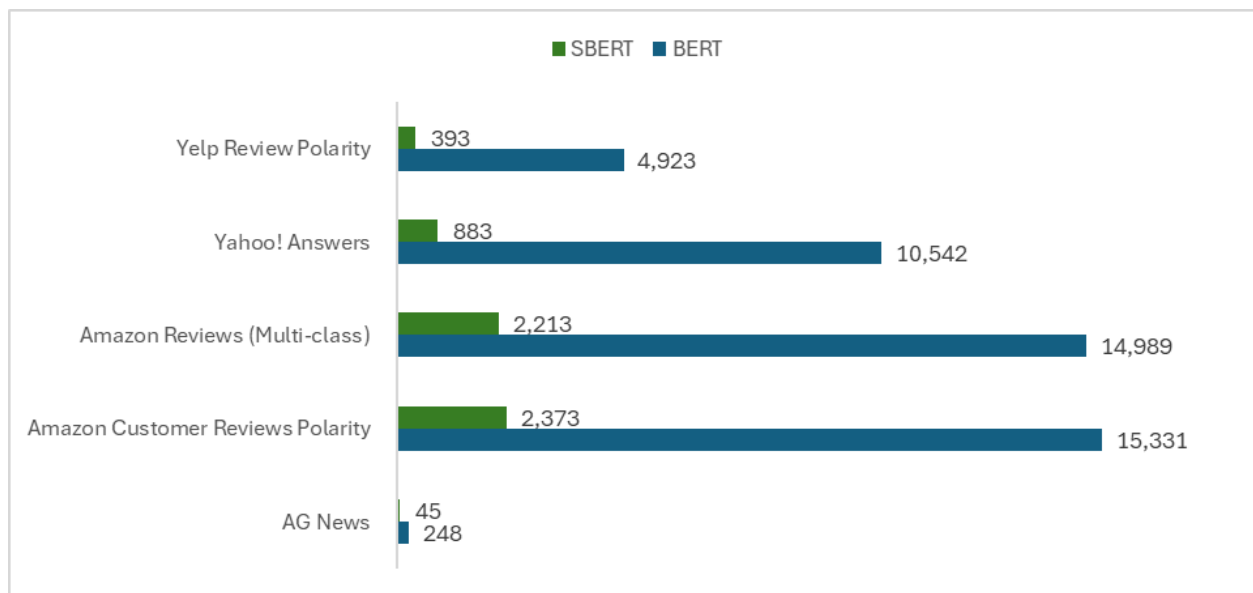
Performance on **Yahoo! Answers** dataset By Model Type



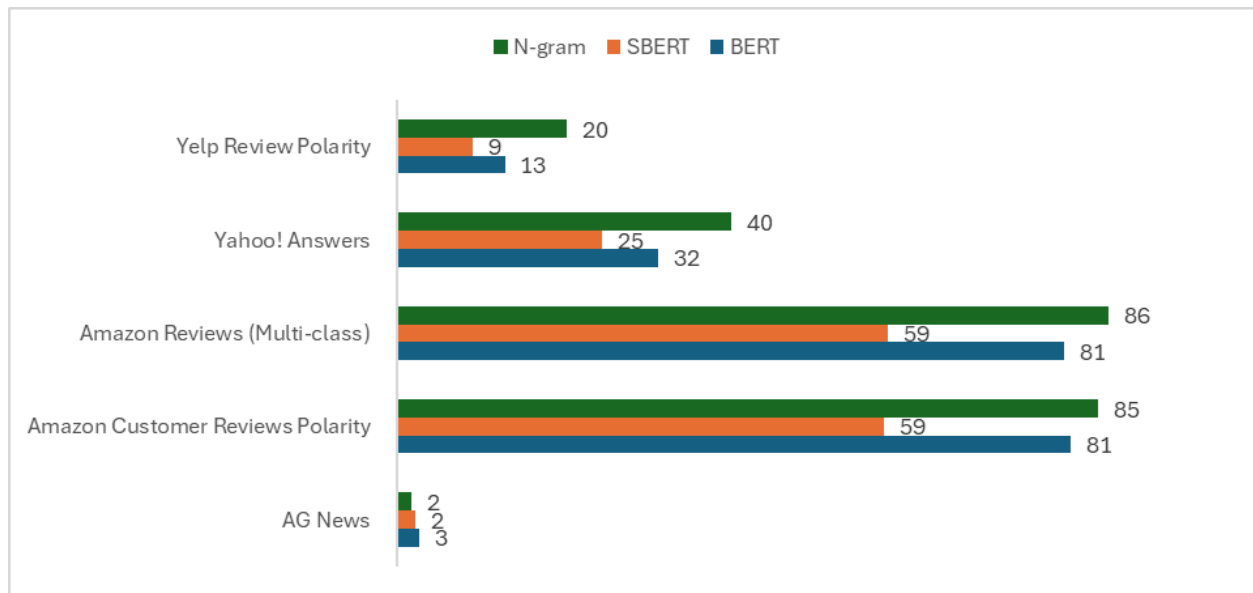
Performance on **Yelp Review Polarity** dataset By Model Type



Embedding time (in seconds) by Model Type



Training time (in seconds) by Model Type



Performance by dataset by model

Dataset	Model Type	Embedding Time (s)	Training Time (s)	Accuracy	Precision	Recall	F1-score
AG News	BERT	248.02	2.67	84.24	84.32	84.24	84.27
Yelp Review Polarity	BERT	4923.24	13	86.74	86.78	86.74	86.73
Yahoo! Answers	BERT	10541.61	31.56	56.98	56.54	56.98	56.6
Amazon Reviews (Multi-class)	BERT	14988.59	80.51	85.76	85.76	85.76	85.76
Amazon Customer Reviews Polarity	BERT	15330.84	81.34	85.74	85.74	85.74	85.74
AG News	N-gram	0	1.74	91.89	91.89	91.89	91.88
Yelp Review Polarity	N-gram	0	20.46	94.91	94.91	94.91	94.91
Yahoo! Answers	N-gram	0	40.32	68.56	68.54	68.56	68.54
Amazon Reviews (Multi-class)	N-gram	0	85.94	93.61	93.61	93.61	93.61
Amazon Customer Reviews Polarity	N-gram	0	84.69	93.58	93.58	93.58	93.58
AG News	SBERT	45.4	2.21	88.66	88.67	88.66	88.66
Yelp Review Polarity	SBERT	393.18	9.15	84.8	84.81	84.8	84.8
Yahoo! Answers	SBERT	882.62	24.75	73.08	72.74	73.08	72.76
Amazon Reviews (Multi-class)	SBERT	2213.21	59.29	79.92	79.92	79.92	79.92
Amazon Customer Reviews Polarity	SBERT	2372.53	58.77	79.92	79.92	79.92	79.92

Average performance across all datasets by Model Type

Model Type	Embedding Time (s)	Training Time (s)	Accuracy	Precision	Recall	F1-score
BERT	9,206	42	79.89	79.83	79.89	79.82
N-gram	NA	47	88.51	88.51	88.51	88.50
SBERT	1,181	31	81.28	81.21	81.28	81.21

